“Amashop:An E-Commerce Web Application for ElectroShop using MERN Stack”

A Project Report submitted in partial fulfillment of the requirement

For the award of degree of

Master of Computer Application
2024-26

Supervised By

Mr. Rohit Sharma

Assistant Professor

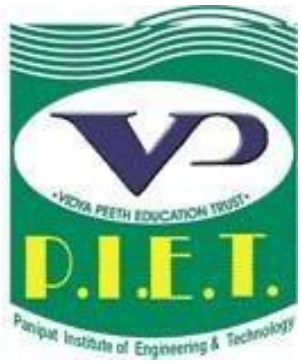
Department of Computer Applications

Submitted By

Garima

MCA 2nd Sem

00240504010019



Department of Computer Applications

PANIPAT INSTITUTE OF ENGINEERING & TECHNOLOGY

2024-2026

Acknowledgement

A successful completion of this project is attributed to the great and indispensable help received from different people.

I will always be grateful and thankful to Head of Department **Dr. Dinesh Verma** and Project Mentor **Mr. Rohit Sharma** for giving me the opportunity to learn the different aspects of designing and implementing this system. It would never be possible for me to design this system without your continuous assistance and guidance.

I want to thanks to my project lead **Mr. Amit** for providing me platform, necessary facilities, guidance and support for completing this project.

I would like to thanks to all faculty members of **Computer Applications Department** who are always encourage me during progress of this project.

Garima

Declaration

I, Garima a student of Master of Computer Applications, in the Department of Computer Application, Panipat Institute of Engineering and Technology, Panipat, under class Roll No. 242023 and University Roll No. 00240504010019 for the Session 2024-2026, hereby, declare that the project report **MCA-283A** entitled "Design and Development of a Secure Blog Platform with User Authentication and Email Verification" has been completed by me in 2nd semester during the two-month project training. I hereby declare that the matter embodied in this project is my original work and has not been submitted earlier for award of any degree or diploma in any college or university.

Date: **Garima**

Department of Computer Applications
Panipat Institute of Engineering and Technology, Samalkha
Certificate

It is certified that Garima, a student of Master of Computer Applications, under class Roll No. 242023 and University Roll No. 00240504010019 for the Session 2024-2026, has completed the project entitled "Amashop: An E-Commerce Web Application for ElectroShop using MERN Stack" under my supervision.

The project report is the authenticate work of the candidate as per her declaration and is found to be fit for the award of degree of Master of Computer Applications at the Panipat Institute of Engineering and Technology, Panipat (affiliated with Kurukshetra University, Kurukshetra).

I wish her all the success in his all endeavors.

Mr Rohit Sharma

Assistant Professor

Department of Computer Applications, PIET

Counter-signed by

HOD-Computer Applications

Certificate from Company

CodeQuotient

Ref. no.: CQ/HR/2508-135 **Dated: 22nd Aug 2025**

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Ms. Garima** student of **Panipat Institute of Engineering & Technology,** Roll no**. 242023 ,** class **MCA** has successfully completed a 6-8 week Industrial training program at CodeQuotient from **July 2025** to **August 2025**.

During this training, she worked under the guidance of **Mr. Ashish Gartan**. Her overall performance during this period was **Satisfactory.**

For CodeQuotient Pvt. Ltd.

Authorized Signatory

CodeQuotient Pvt Ltd, Plot No 12A, HSIIDC IT Park, Sector 22, Panchkula, Haryana
info@codequotient.com | https://codequotient.com/

Table of Contents

Chapter 1
Company Profile

Chapter 1 **Company Profile**

1.1 About Company

Code Quotient is on a mission to make India the talent capital of the world by unlocking the untapped potential of aspiring engineers. Founded by Arun Goyat (Founder & CEO), Code Quotient bridges the gap between academia and industry through practical, project-based learning. With a strong focus on Artificial Intelligence, software engineering, and emerging technologies, the company prepares students and professionals to excel in global tech ecosystems..

1.2 Our Vision

To transform India's vast student population into world-class software and AI engineers by nurturing talent through real-world problem-solving, expert mentorship, and accessible education. We aim to bridge the gap between academic knowledge and industry needs by providing students with hands-on exposure to cutting-edge technologies, collaborative projects, and career-focused training. Our vision is not just to create skilled professionals but to empower innovators, leaders, and change-makers who will drive India's growth in the global technology landscape.

1.3 Key Initiatives

1. **CQ AI Program** – A 3-month, zero-cost, intensive program where selected students sharpen their AI engineering skills, work on real-world projects in NLP, Computer Vision, and Generative AI, and receive mentorship from experienced AI engineers.
2. **CodeQuotient School of Technology (CQST)** – India's first AI-exclusive higher education institute. CQST redefines traditional education with UGC-recognised and AICTE-approved degree programs combined with industry-driven training, real-world AI applications, and internships.
3. **University Partnerships** – CodeQuotient enables universities to integrate industry-ready AI education into their curriculum through:

- AI-powered Learning Management System (LMS)
- Automated assessments and exams
- Centers of Excellence in AI
- Industry-trained expert faculty

4. **Talent Development & Hiring** – Partner companies access highly trained talent pools, especially from Tier II/III cities, aligning with their long-term hiring needs.

1.4 What sets us apart

- Not just training, but **real-world experience** through hands-on projects and open-source contributions.
- **Zero-cost opportunities** for deserving candidates, ensuring financial barriers do not block talent.
- **Strong industry integration** with internships, hackathons, and mentorship that mirror real workplace challenges.
- **Fee sponsorships** and career readiness programs supported by hiring partners.

Chapter 2
Introduction

2.1 Introduction

In the modern digital era, e-commerce has transformed the way people purchase products by offering the convenience of shopping online. With the increasing use of technology, businesses are moving towards web-based platforms to provide better accessibility and reach a wider audience.

This project, **Amashop**, is an e-commerce web application developed for **ElectroShop**, an online electronics store. The platform allows customers to **browse electronic products, view details, and add them to the shopping cart** for easy management.

The application is built using the **MERN stack (MongoDB, Express.js, React.js, and Node.js)**, ensuring a dynamic, scalable, and user-friendly shopping experience. The backend manages user authentication, product handling, and cart functionalities, while the frontend provides an interactive and responsive interface.The main goal of this project is to build a reliable online store that simplifies product browsing and cart management while demonstrating the practical implementation of full-stack web development concepts.

2.2 Purpose

- Reduce the gap between sellers and buyers by providing a digital marketplace.
- Demonstrate the implementation of **MERN stack (MongoDB, Express.js, React.js, Node.js)** in building a real-world application.
- Provide hands-on experience in **full-stack development**, integrating frontend, backend, and database.

2.3 Objectives

- **To develop an online platform** where users can browse and view different electronic products.
- **To integrate backend services** using Node.js and Express.js for managing APIs and server-side logic.

Chapter 3
Feasibility Study

Chapter 3 Feasibility Study

3.1 Overview

The feasibility study is an essential part of every project as it helps in analyzing whether the project is practically possible and sustainable in the long run. In the case of the **Amashop (ElectroShop)** e-commerce website, the feasibility study has been carried out to ensure that the design, development, and deployment of the system can be achieved without major obstacles. The system has been designed with three main modules – the **User Panel, Seller Panel, and Admin Panel** – each serving a unique purpose. The user panel allows customers to register or log in to their account, explore electronic products, and add them to their cart for future purchase decisions. The seller panel enables sellers to showcase their products by adding new items, updating product details, or removing items that are no longer available. The admin panel, on the other hand, is the backbone of the entire system as it provides control and supervision over all users and sellers, making the platform secure, transparent, and organized. This feasibility analysis is carried out to check technical capabilities, economic requirements, operational efficiency, and scheduling aspects, ensuring that the project can be implemented successfully with the available resources and within the given timeframe.

Preliminary investigation of any project is its Feasibility Study. This report provides overview of all relevant data. There are five major dimensions of feasibility study acronym as TELOS: Technical, Economic, Legal, Operational and Scheduling. If these five dimensions are successfully tested for adding new modules and debugging existing running system.

The types of feasibility study that are covered during preliminary investigation:

- Technical Feasibility
- Economic Feasibility
- Legal Feasibility

3.2 Technical Feasibility

From a technical perspective, the Amashop project is highly feasible because it uses the modern and powerful MERN stack (MongoDB, Express.js, React.js, Node.js). This stack is well-suited for building dynamic, interactive, and scalable web applications. React.js is employed in the frontend, which provides an attractive, responsive, and user-friendly interface for the customers, sellers, and admin. Node.js and Express.js handle the backend operations efficiently, offering smooth request handling, session management, and routing. The database is designed in MongoDB, a NoSQL database that provides high scalability and flexibility for storing structured and unstructured data such as user details, product information, and transaction logs. Security is ensured through JWT authentication which prevents unauthorized access to sensitive parts of the website, such as the seller's product dashboard and the admin's monitoring panel. The technology stack used is open-source, cost-effective, and widely supported, making it reliable and future-proof. Additionally, the system can be deployed on cloud platforms like MongoDB Atlas for database hosting, Render or Heroku for backend deployment, and Netlify or Vercel for frontend hosting, thereby ensuring smooth performance and easy accessibility. Considering these factors, the technical feasibility of the project is strong, and no major risks are foreseen.

3.3 Key Technical Feasibility Questions

To further validate the technical feasibility, some critical questions were considered during the planning and development stages. Will the system support a large number of users simultaneously? The answer is yes, because React.js and Node.js provide scalability and MongoDB handles large datasets efficiently. Will authentication and authorization be secure? This concern has been addressed by implementing JWT tokens, ensuring that only authorized users, sellers, and admins can access their respective panels. Will the project's chosen technology stack remain relevant in the future? The MERN stack is currently one of the most popular choices for full-stack development and has a vast developer community, which guarantees continuous updates and long-term support. Will integration between the user panel, seller panel, and admin panel be smooth? Since all three modules are connected.

3.4 Economic Feasibility

The system is economically feasible because it is developed using open-source technologies that do not require expensive licenses. The cost of development is minimal as it relies entirely on free frameworks and libraries. Hosting expenses are low since cloud-based services provide affordable solutions for deployment. Maintenance costs are also limited due to the modular design of the application, which makes debugging and future updates easier. In return, the project offers significant benefits, such as providing a seamless shopping experience for users, a convenient product management platform for sellers, and full control and monitoring capabilities for the admin. The balance between low cost and high benefits makes the project economically viable.

3.5 Operational Feasibility

The system is operationally feasible as it has been designed with simplicity and user-friendliness in mind. The user panel provides customers with an easy-to-navigate shopping environment where they can quickly log in, browse products, and manage their cart. The seller panel simplifies the process of product management, even for individuals with limited technical knowledge. The admin panel ensures complete control over the system by allowing the administrator to monitor and manage both users and sellers effectively. The website does not require any special training for its users, and updates or bug fixes can be smoothly integrated. The overall design and workflow ensure that the system is practical and efficient in real-world operations.

3.6 Scheduling Feasibility

The project development has been planned in a structured way, making it feasible to complete within a reasonable time frame. The initial stages include requirement gathering and analysis, followed by the design of the user interface and database structure. Once development is complete, the system undergoes thorough testing and debugging to ensure error-free performance. Finally, the deployment and documentation stages are executed. The entire process can be completed within six to eight weeks, making the scheduling feasible and achievable within academic timelines.

Chapter 4
Requirement Analysis

Chapter 4 Requirement Analysis

4.1 Requirement Analysis

Requirement analysis is one of the most critical phases in the software development life cycle, as it helps in understanding the expectations of stakeholders and the functional as well as non-functional needs of the system. In the case of the **Amashop (ElectroShop)** project, requirement analysis was conducted to identify the exact needs of three important modules: the user panel, the seller panel, and the admin panel. Users are the core of the system, and they require features such as registration, login, product exploration, and the ability to add items to their cart. Sellers, on the other hand, need functionality for adding products, updating product details, and deleting products when required. The admin panel acts as the controlling unit of the system, where the administrator monitors user activities, seller operations, and ensures that the platform remains secure and efficient. The analysis was carried out using structured methods such as discussion, brainstorming, and case studies of existing e-commerce websites. The findings clearly highlighted that the system should be simple, interactive, secure, and scalable. This analysis laid the foundation for the design and implementation stages, ensuring that the developed system will meet real-world expectations effectively.

4.2 Software Requirement

The Amashop system has been designed using the **MERN stack** which is one of the most powerful and widely used combinations for full-stack web development. On the frontend, **React.js** has been employed to build an attractive, responsive, and user-friendly interface that allows smooth navigation for customers, sellers, and administrators. On the backend, **Node.js with Express.js** has been used to handle server-side logic, API endpoints, authentication, and communication with the database. For storing data such as user details, seller information, product catalog, and admin records, **MongoDB** has been chosen due to its flexibility, scalability, and NoSQL schema-less design.

- Frontend: React.js for responsive UI.
- Backend: Node.js + Express.js for API and logic.
- Database: MongoDB for scalable storage

4.3 Hardware Requirements

The hardware requirements for this project are minimal and affordable since it is a web-based system. For development purposes, the project requires a computer system with at least an **Intel i3 processor or equivalent**, **8 GB RAM**, and a **minimum of 250 GB of hard disk space**. A stable internet connection is required to interact with external libraries, APIs, and hosting platforms. For running the server locally, **Node.js runtime** is required, and MongoDB can either be installed locally or accessed through **MongoDB Atlas**.

- Minimum Intel i3, 8 GB RAM, 250 GB storage.
- Internet connection required for deployment.
- Runs smoothly on laptops, desktops, tablets, or smartphones.

4.4 Functional Requirements

Functional requirements describe the core operations and activities that the system must perform. For the **user panel**, the functional requirements include user registration, login/logout, browsing the product catalog, searching for products, and adding selected items to the cart. The **seller panel** includes features such as login authentication, adding new products with descriptions and prices, updating product details, and deleting unavailable products from the list. The **admin panel** contains the most critical functional requirements, as it allows the administrator to monitor user and seller activities, manage the product database, and ensure overall platform security.

- User Panel: Login, logout, view products, add to cart.
- Seller Panel: Add, update, delete products.
- Admin Panel: Monitor activities, approve/remove products.

4.5 Non-Functional Requirements

Non-functional requirements focus on the qualities and constraints of the system rather than specific functions. In the case of Amashop, **security** is a primary non-functional requirement, achieved through JWT authentication, encrypted passwords, and secure database management. **Scalability** is another requirement, as the system should be able to handle an increasing number of users, sellers, and products in the future without performance degradation. **Usability** is ensured by designing a clean and intuitive interface using React.js, so even non-technical users can easily operate the system. **Reliability** is considered through proper backend design that minimizes downtime and errors.

- Security through JWT and encryption.
- Scalability to support growing users and sellers.
- Usability with a simple and clean interface.

Chapter 5
System Design

Chapter 5 System Design

5.1 Overview

System design is one of the most important phases of software development because it transforms the requirements identified in the analysis stage into a structured solution. In the case of the **Amashop (ElectroShop)** project, system design defines how different components such as the user panel, seller panel, and admin panel interact with each other and how data flows across the application. The objective of system design is to provide a blueprint that ensures scalability, reliability, and maintainability. It also determines the architecture of the system, including the database schema, entity relationships, and interaction between frontend and backend.

System design = Blueprint for development, covering structure, database, and workflows.

5.2 Entity-Relationship Diagram

The ERD illustrates the logical structure of the database used in Amashop. It shows how different entities such as **Users, Sellers, Products, Admin, and Cart** are connected. For example, a user can register and maintain a cart, while sellers manage products. The admin has privileges to monitor and control both users and sellers.

Entities in the ERD:

- **User**: UserID, Name, Email, Password.
- **Seller**: SellerID, Name, Email, Products.
- **Product**: ProductID, Name, Price, Category, SellerID.
- **Admin**: AdminID, Name, Email, Role.

5.3 Why Make an ERD?

Creating an ERD provides a clear representation of the data model before implementation. It helps developers and database designers visualize the relationships between entities, avoid redundancy, and maintain consistency. In the Amashop project, the ERD ensures that the interaction between **user activities, seller operations, and admin monitoring** is properly captured at the database level.

- Simplifies database design.
- Avoids data duplication and maintains integrity.
- Acts as documentation for developers.

5.4 Components of ER Diagram

ERD is basically consists of five main components:

Features	Description
Entity	Entity is anything real or concept or abstract about which we can store data. Those have some information are entities. It is represented as a rectangular box. Entities of the proposed system are: admin,employee,roll,bale,etc.
Relationship	Relationship is an action which defines how entities will share the data. It is represented by diamond shape. It acts as an association between entities. e.g. Create roll
Cardinality	Defines how instances of entities are related to other entities. It also defines the number of instances of entity relate to instance of another entity. It basically specifies the occurrences of entities. e.g. Create roll, print it and dispatch
Attribute	Attributes are the characteristics of any entity. It may be common to all or most instances of a particular entity. Attribute can be said as field, data entry. e.g. username, barcode, rolllid, baleid, etc.

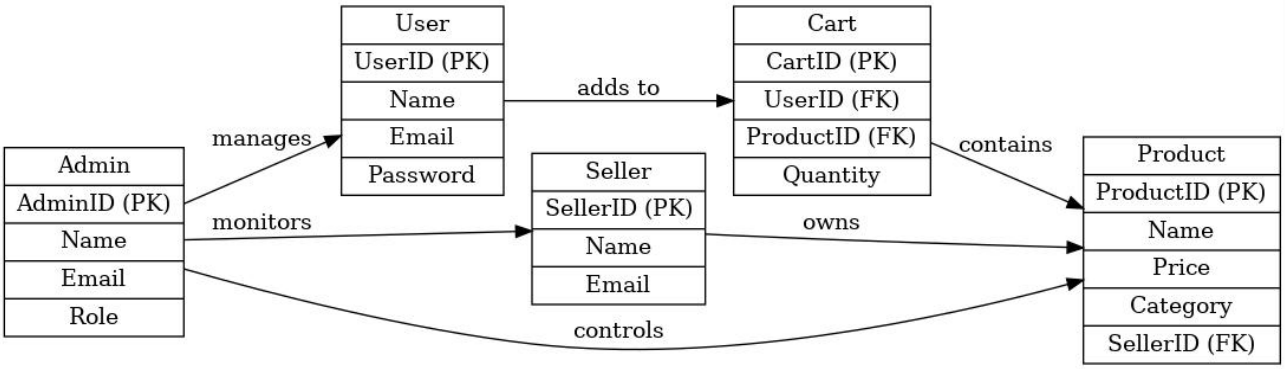


Figure 5.2: ER Diagram

5.4 Data Flow Diagram

Data Flow Diagram was developed by LARRY CONSTANTINE. These were popularized in late 1970s. This is the method to show the system requirements graphically.

It is a traditional visual representation of the flaw of information. It can be designed manually, automated or combination of both. They can be used to analyze an existing system or model new one. DFD is can be divided into Logical and Physical. Logical DFD represents the flow of data into the system whereas Physical DFD represents how the flow of information will be implemented.

DFD is used or designed for:

- To determine the logical flow of the information
- To determine the implementation of logical flow
- To determine the physical construction of the system
- To arrange the system requirements in a format so that system will easily be determined

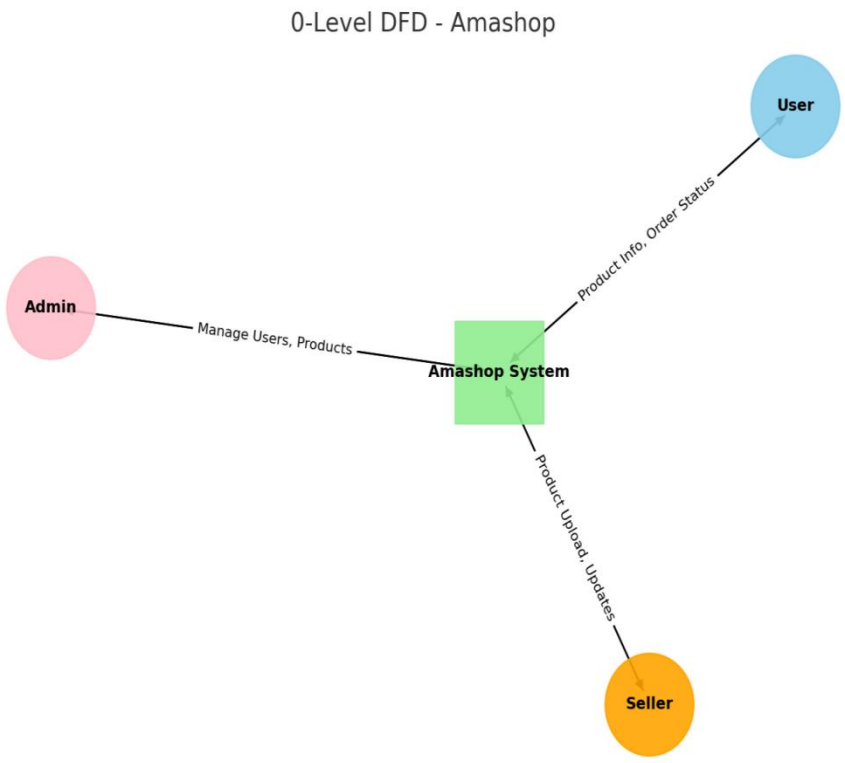
These are some reasons why do we need a DFD in system/project development or software engineering. A clear and neat DFD is helpful to develop an efficient and quantifiable system.

Data Flow Diagram is categorized mainly into these levels:

- 0-Level DFD
- 1-Level DFD

5.5.1 0-Level DFD

Level 0 DFDs, also known as context diagrams, are the most basic data flow diagrams. They provide a broad view that is easily digestible but offers little detail. Level 0 data flow diagrams show a single process node and its connections to external entities.

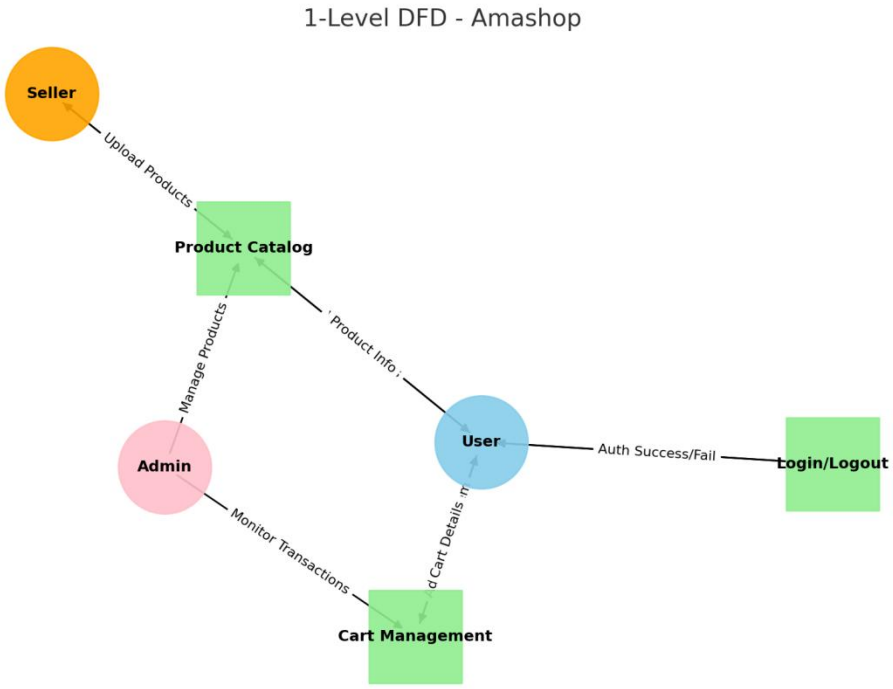


5.5.2 1-LEVEL DFD

In this we process the process of level 0 DFD and expand it. Level 1 DFD that shows some of the detail of the system being modeled. The Level 1 DFD shows how the system is divided into sub-systems (processes), each of which deals with one or more of the data flows to or from an external agent, and which together provide all of the functionality of the system as a whole. It also identifies internal data stores that must be present in order for the system to do its job, and shows the flow of data between the various parts of the system.

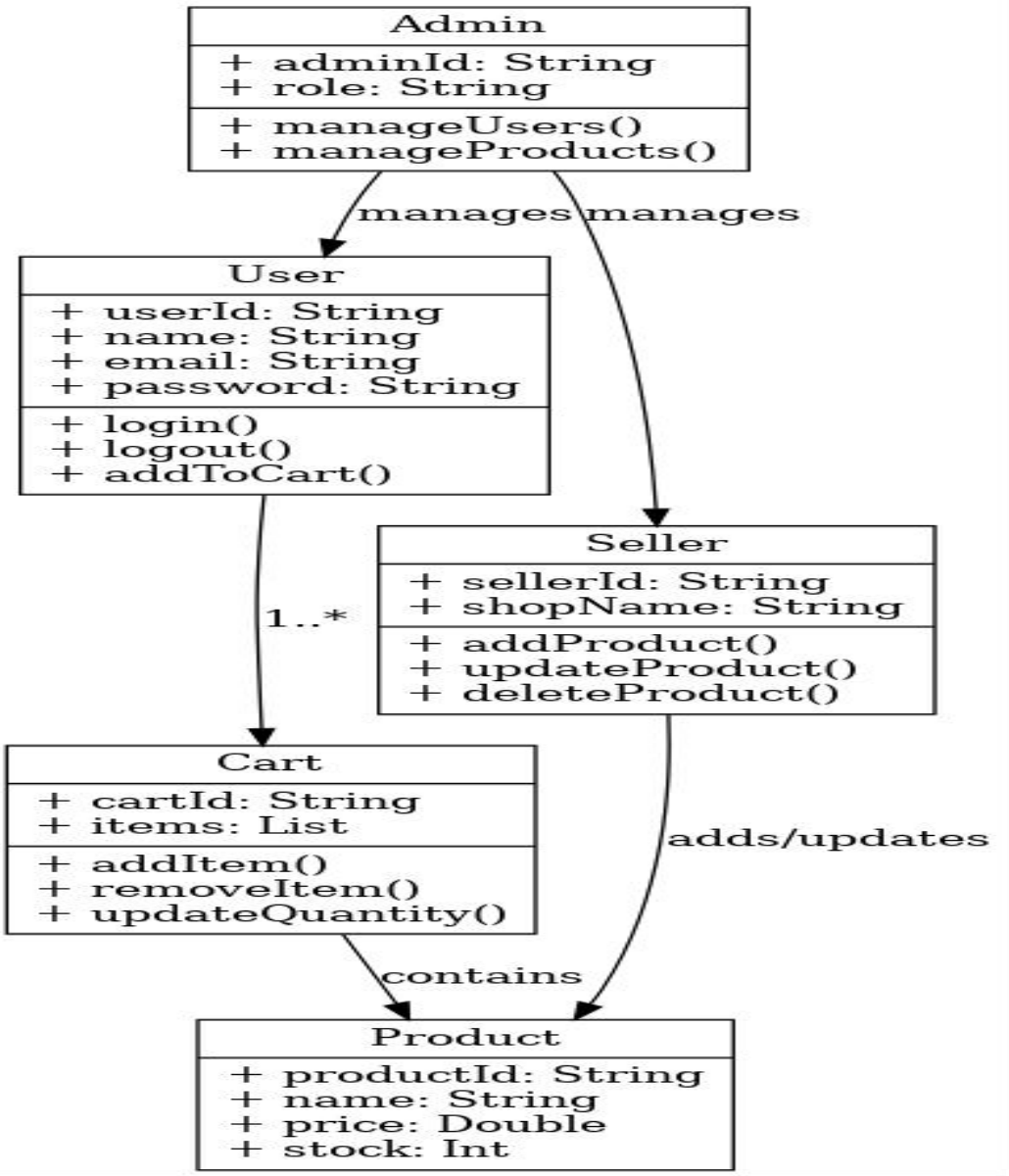
This level DFD states the inner process of each module.

How these processes take place with the help of entities include in it.



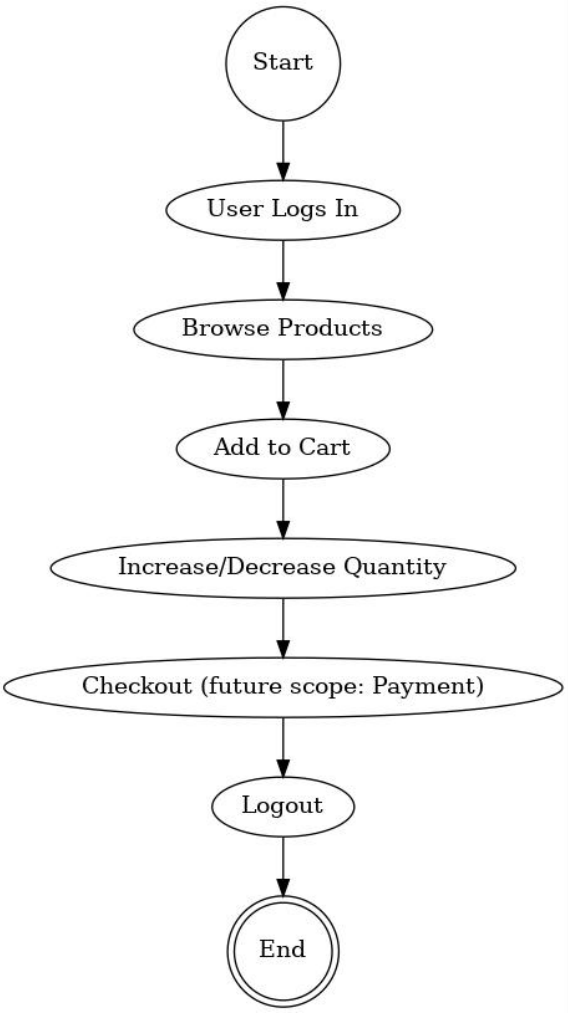
5.6 Class Diagram

The Class Diagram shows the object-oriented view of the Amashop project. It defines classes, their attributes, methods, and relationships



5.7 Activity Diagram

Activity diagrams are graphical representations of workflows of stepwise activities and actions with support for choice, iteration and concurrency. In the Unified Modeling Language, activity diagrams can be used to describe the business and operational step-by- step workflows of components in a system. An activity diagram shows the overall flow of control.



.`

Chapter 6
Testing

Chapter 6 Testing

6.1 Introduction

Software testing is a systematic process used to evaluate and verify that a software application or system meets its specified requirements and works as expected. It involves identifying bugs, errors, or defects in the software to ensure that the final product is reliable, performs correctly under various conditions, and satisfies user needs.

The primary goal of software testing is not just to find faults but also to confirm that the product meets both technical and business requirements. Testing can be done manually or automated using specialized tools, and it happens throughout the software development life cycle. This process includes activities like planning what to test, designing test cases, executing those cases, and tracking defects to make the software as robust as possible.

By ensuring software quality and functionality, testing helps minimize future risks, reduces costs related to fixing issues post-release, and increases overall user satisfaction.

6.2 Objective

The main objectives of software testing are to:

- **Identify Defects**: Detect as many bugs, errors, or issues in the software as early as possible to improve overall quality.
- **Verification and Validation**: Ensure the software meets both user requirements and functional specifications, confirming that it is fit for its intended purpose.
- **Defect Prevention**: Help the team analyze and avoid the root causes of defects so similar issues do not arise in the future.
- **Ensure Quality Attributes**: Validate various quality attributes such as functionality, performance, security, usability, and compatibility of the software.
- **Risk Management**: Mitigate risks by uncovering potential security vulnerabilities, reliability concerns, or business risks before the software is released.
- **Reduce Cost and Time**: Testing early and thoroughly reduces the cost and time associated with fixing bugs that might be discovered after release.

  Verify Compliance: Ensure that the software adheres to legal, regulatory, and industry

6.3 Types of Testing:

- Functional Testing
- Non-Functional Testing

6.3.1 Functional Testing

Functional testing is a type of testing that seeks to establish whether each application feature works as per the software requirements. Each function is compared to the corresponding requirement to ascertain whether its output is consistent with the end user's expectations. The testing is done by providing sample inputs, capturing resulting outputs, and verifying that actual outputs are the same as expected outputs.

Types include:

- Unit Testing: Tests individual components or functions.
- Integration Testing: Checks interactions between modules.
- System Testing: Verifies the complete, integrated system.
- Acceptance Testing: Confirms software meets business/user needs.

6.3.2 Non-Functional Testing

Non-functional testing focuses on evaluating the system's performance, scalability, security, usability, and reliability, rather than its specific functionality. It ensures that the system can handle real-world demands and provides a seamless, high-quality user experience.

Types include:

- Performance Testing: Measures speed, scalability, and stability.
- Load Testing: Checks system behavior under expected user load.
- Stress Testing: Evaluates limits under extreme conditions.
- Security Testing: Finds vulnerabilities.
- Usability Testing: Gauges user-friendliness.
- Compatibility Testing: Checks software on various devices, OS, browsers.

6.4 Testing Approaches

Manual Testing

Automated Testing

6.4.1 Manual Testing

Manual testing is a process where QA engineers manually test a software application to find bugs by following a written test plan with defined scenarios. They compare actual behavior against expected results and report any issues. It ensures the software meets user expectations, works correctly, and provides a good user experience. Manual testing is especially useful in early development and exploratory testing, where human intuition and adaptability are important.

Types of Manual Testing:

- **Unit Testing**

User testing involves the verification of individual components or units of source code. A unit can be referred to as the smallest testable part of any software. It focuses on testing the functionality of individual components within the application. Developers often use it to discover bugs in the early stages of the development cycle.A unit test case would be as fundamental as clicking a button on a web page and verifying whether it performs the desired operation. For example, you are ensuring that a share button on a webpage lets you share the correct page link.

- **Integration Testing**

Integration Testing is the next step after unit testing. Multiple units are integrated to be tested as a whole. For example, testing a series of webpages in a particular order to verify interoperability. This approach helps QAs evaluate how several application components work together to provide the desired result. Performing integration testing in parallel with development allows developers to detect and locate bugs faster.

- **System Testing**

System testing involves testing all the integrated modules of the software as a whole. It helps QAs verify whether the system meets the desired requirements.

6.4.2 Automated Testing

Automation testing is a technique that uses tools and scripts to automatically run tests, check results, and manage repetitive tasks with little human effort. Its main purpose is to improve efficiency, accuracy, and test coverage compared to manual testing. It is especially effective for regression testing, load testing, and large projects where repeated execution is required. Automation also helps speed up development cycles and ensures faster delivery of reliable software.

Types of automation testing:

- **Smoke testing:** Smoke testing is a type of software testing that determines whether the built software is stable or not. It is the preliminary check of the software before its release in the market.
- **Performance testing:** Performance testing is a type of software testing that is carried out to determine how the system performs in terms of stability and responsiveness under a particular load.
- **Regression testing:** Regression testing is a type of software testing that confirms that previously developed software still works fine after the change and that the change has not adversely affected existing features.
- **Security testing:** Security testing is a type of software testing that uncovers the risks, and vulnerabilities in the security mechanism of the software application. It helps an organization to identify the loopholes in the security mechanism and take corrective measures to rectify the security gaps.
- **Acceptance testing:** Acceptance testing is the last phase of software testing that is performed after the system testing. It helps to determine to what degree the application meets end users' approval.
- **API testing:** API testing is a type of software testing that validates the Application Programming Interface(API) and checks the functionality, security, and reliability of the programming interface.
- **UI Testing:** UI testing is a type of software testing that helps testers ensure that all the fields, buttons, and other items on the screen function as desired.

Chapter 7
Implementation

7.1 Software Used:

The implementation of the ***Amashop*** E-commerce website required a range of software tools to ensure smooth development and deployment. The backend of the system was implemented using **Node.js** with **Express.js**, which provided a fast and reliable server environment. For the frontend, **React.js** was used to build an interactive and responsive user interface. **MongoDB** served as the database to store product details, user information, and cart data. Additionally, **Visual Studio Code** was the primary IDE used for coding, while **Postman** was employed for testing APIs during development. Version control was managed using **Git and GitHub**, which allowed proper tracking and collaboration.

7.2 Hardware Specification:

The implementation did not require very high-end hardware since the project is lightweight and web-based. The development and testing were carried out on a standard laptop with the following specifications:

- **Processor**: Intel Core i3
- **RAM**: 8 GB
- **Storage**: 500 GB HDD/SSD

Display: 14” or higher resolution display
This configuration was sufficient for running the Node.js server, database, and frontend development simultaneously without performance issues.

7.3 Operating System :

Window 10 .

7.4 Languages Used:

Following technologies that are used are :

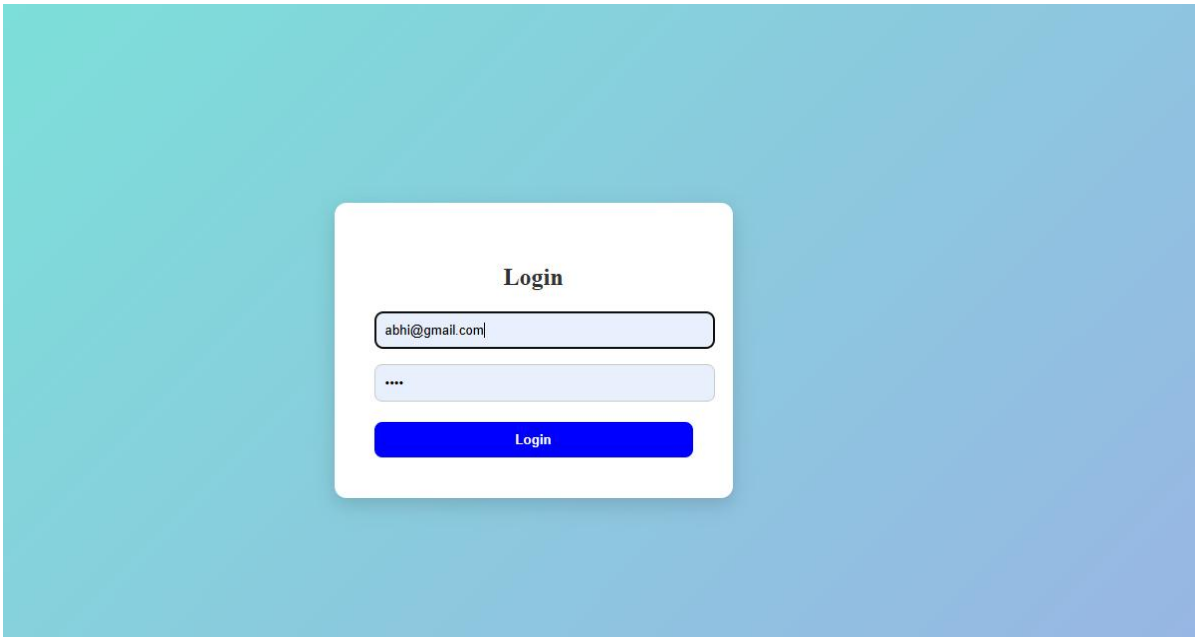
- JavaScript
- React
- Tailwind CSS and HTML

Chapter 8
Snapshots

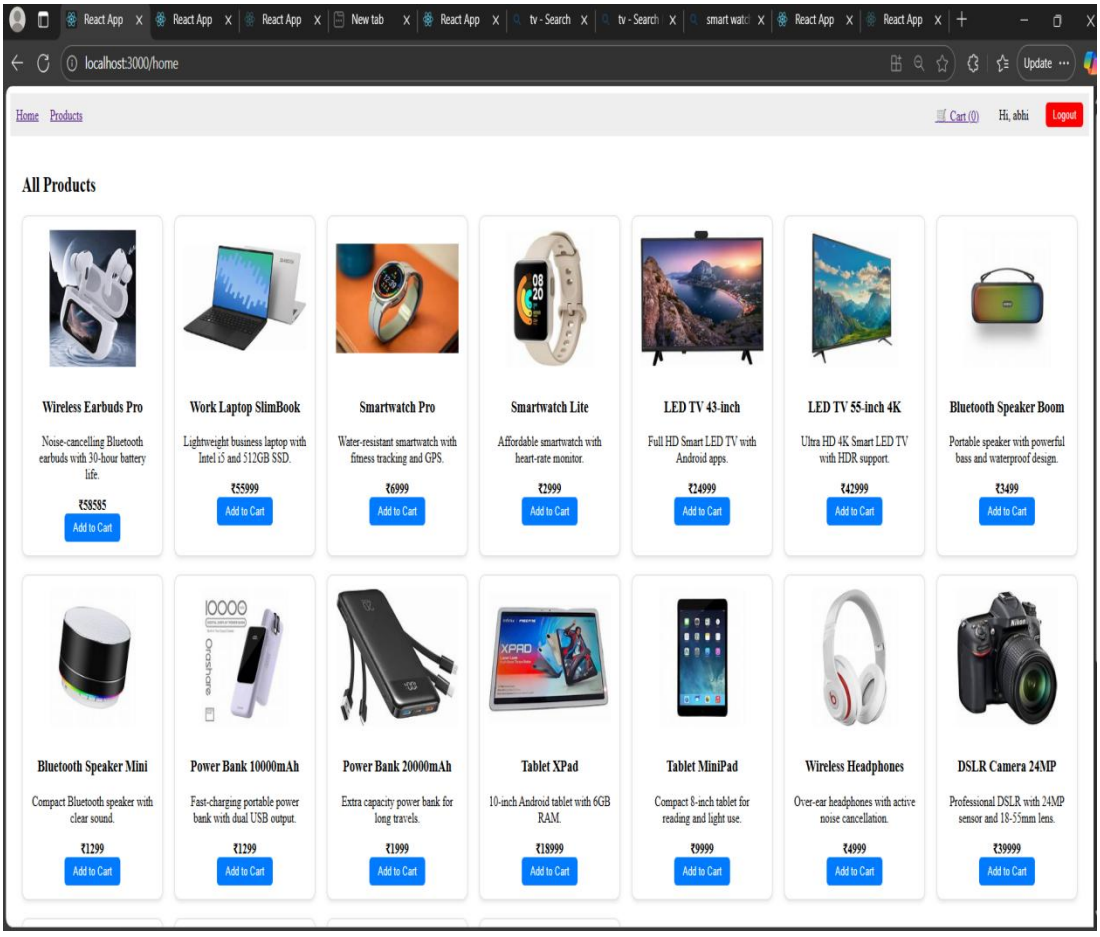
8.1 Login Page

The login page is an essential module of our project. It allows registered users, such as customers, sellers, and administrators, to securely access their accounts by entering valid credentials (email/username and password).

This page is designed with a simple and user-friendly interface, ensuring that users can easily log in without confusion. It also provides validation to prevent unauthorized access, displaying error messages if the entered details are incorrect.



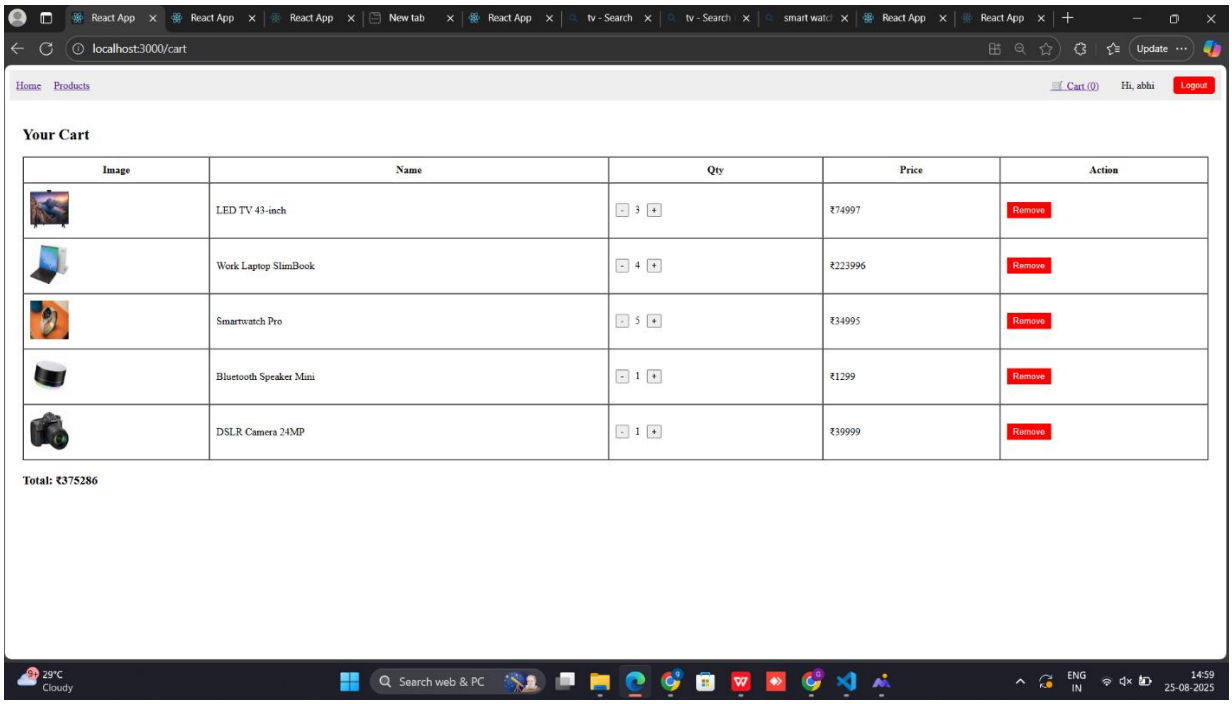
8.2 Home Page



The home page is the main entry point of the application and provides an overview of the platform. It is designed to be simple, attractive, and user-friendly so that users can easily navigate to different sections such as products, categories, cart, and user account.

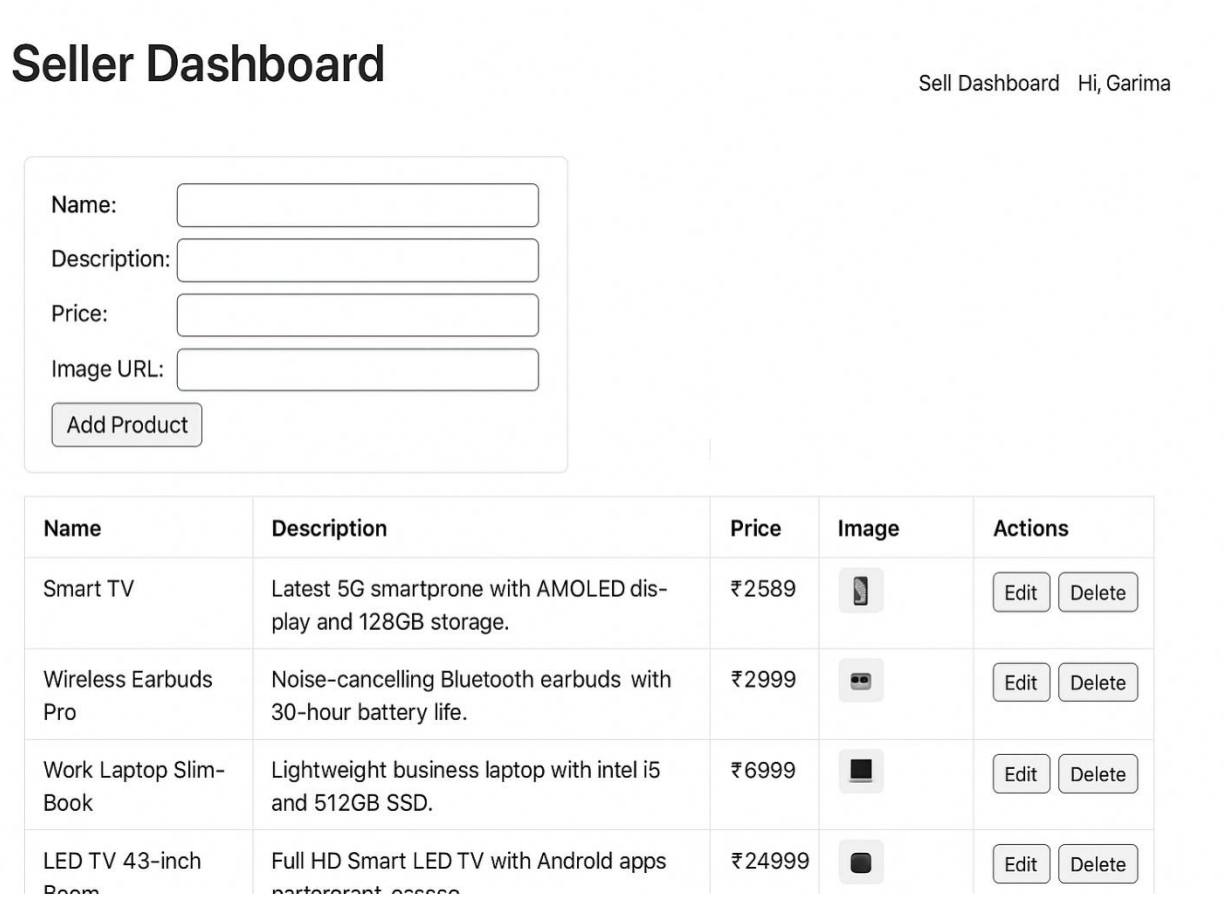
8.3 Cart Page

The cart page allows users to review the products they have selected before proceeding to checkout. It displays detailed information such as product name, quantity, price, and total amount. Users can update the quantity of items, remove products, and view the overall bill including taxes and shipping charges.



8.4 Seller Dashboard

The seller dashboard is designed to help sellers manage their products and orders efficiently. It provides a centralized panel where sellers can add new products by entering details such as product name, description, price, and image URL. Sellers can also view the complete list of their products with options to edit or remove them.



8.4 Admin Dashboard

Admin Dashboard

HI, garima9813 Legout

Registered Users

Name	Email	Role	Admin
Abhi	abhi@seller.com	User	User
Garima	garima9813@gmail.com	User	User
Atul	atul@gmail.com	Admin	Admin

Manage Products

Name | Price | Description (optional) | Add

Image	Name	Price	Description	Category
	Smartphone X200 Latest 5G	₹2999	Smartpri&mab Pro	Electronics
	Wireless Earbuds Pro Smartwatch Pro	₹2999	Workess Sin%ImiSoik	Electronics
	Work Laptop SlimBook Smartwatch	₹5599	Smartwatch Pro	Electronics
	Smartwatch Lite LED TV 43–inch	₹6999	Smartwatch Lite	Electronics
	LED TV 55–inch 4K LED TV 55–inch 4K	₹5999	LED TV 55–inch	Electronics
	Bluetooth Speaker Boom Bluetooth Spea	₹9999	Bnee19ank Speaker Mini	Electronics
	Power Bank:10000mAh Blueefooth Pro	₹9999	Power Bank 20000mAh	Electronics
	Tablet XPad Tablet MinIPad	₹3999	Tablet XPad	Electronics
	Wireless Headphones DSLR Cantera 24MP	₹6999	Wireless Headphones	Electronics
	Mirrorless Camera 26MP Kitrollieaf*SLIDVEL	₹5999	Wireless Keyboard & Mouse	

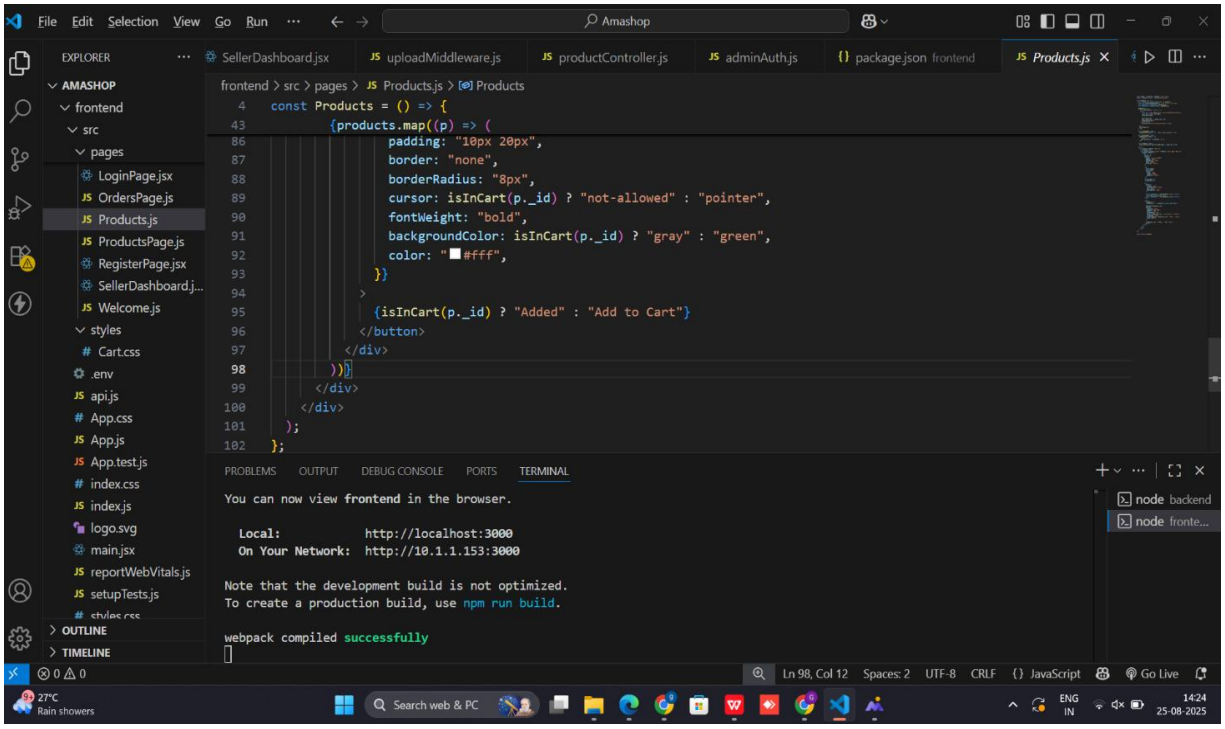
User Orders (Cari)

User	Email	Products	Total Price
garima9813@gmail.com	garima9813@gmail.com	Smartphone X200	₹32,000
abhi@seller.com	abhi@seller.com	Wireless Earbuds Pro	₹27,000

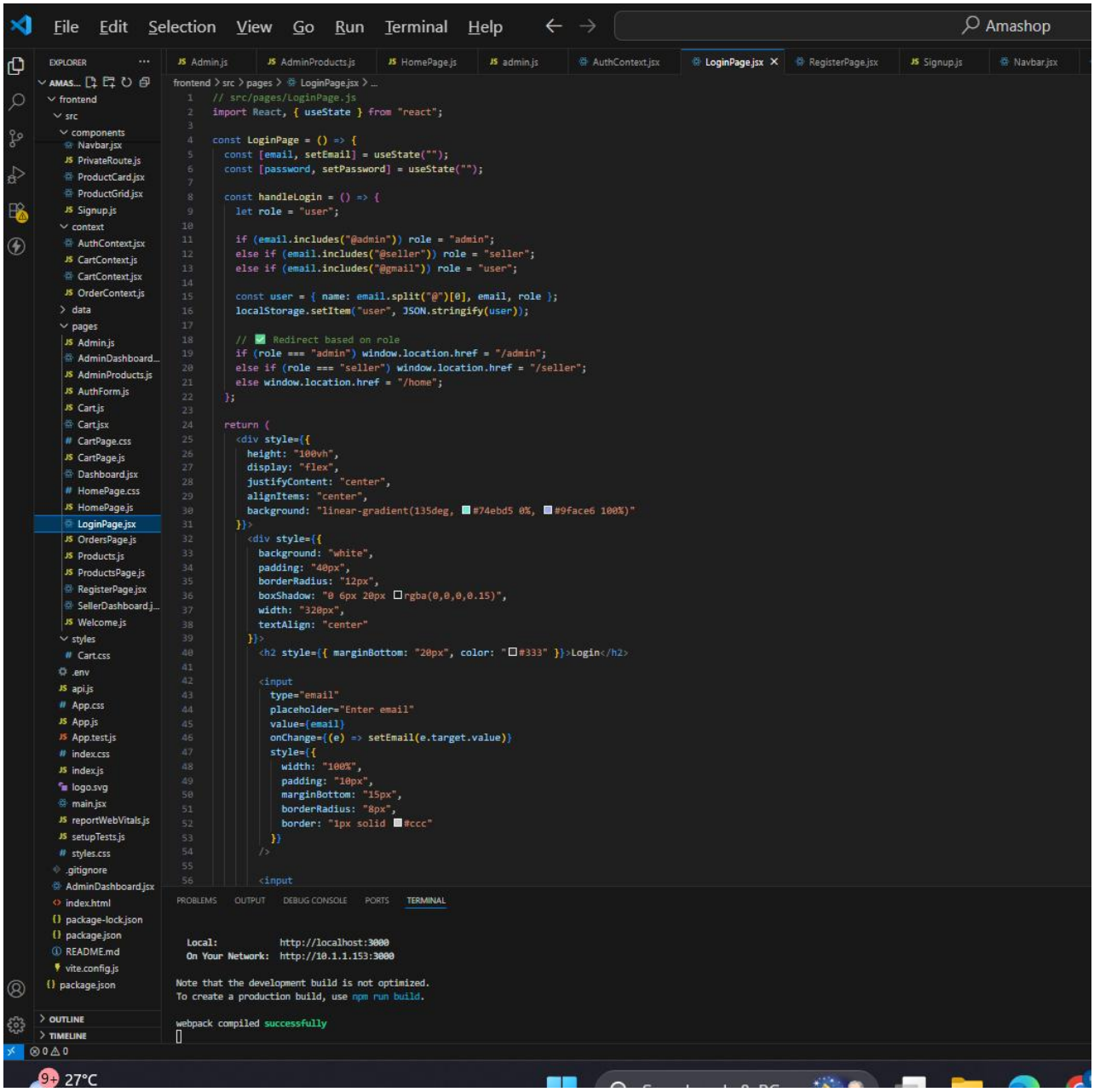
Chapter 9
FRONTEND,BACKEND & DATABASE SNAPSHOTS

9.1 Frontend Folder Structure:

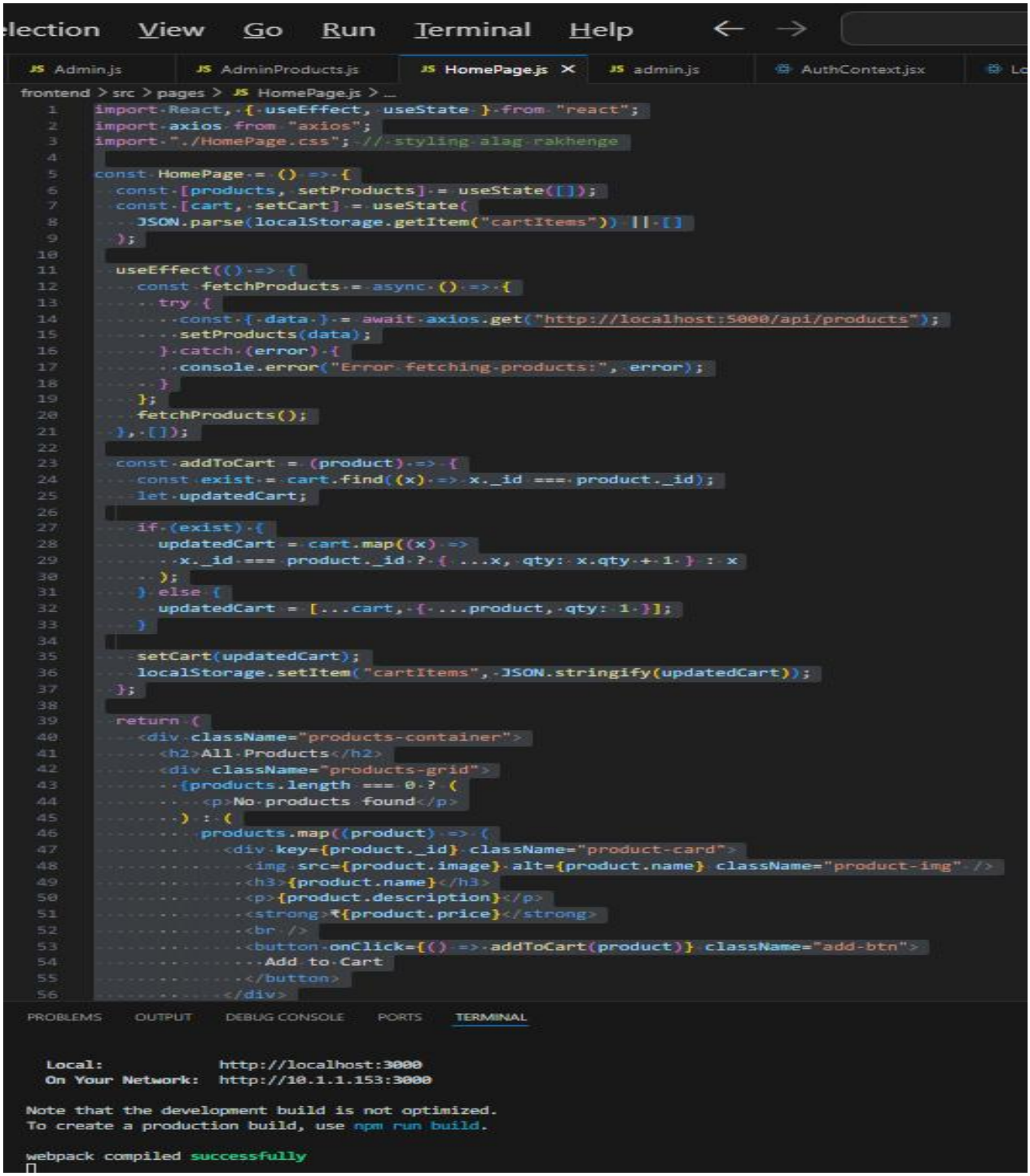
This screenshot shows the structure of frontend directory.



9.2 Login Page



9.3 Home Page


lection View Go Run Terminal Help
Admin.js
AdminProducts.js
HomePage.js
admin.js
AuthContext.jsx
frontend > src > pages > HomePage.js > ...
import React, { useEffect, useState } from "react";
import axios from "axios";
import "./HomePage.css"; //styling alag rakhenge
const HomePage = () => {
const [products, setProducts] = useState([]);
const [cart, setCart] = useState(
JSON.parse(localStorage.getItem("cartItems")) || []
);
useEffect(() => {
const fetchProducts = async () => {
try {
const { data } = await axios.get("http://localhost:5000/api/products");
setProducts(data);
} catch (error) {
console.error("Error fetching products:", error);
}
};
fetchProducts();
}, []);
const addToCart = (product) => {
const exist = cart.find((x) => x._id === product._id);
let updatedCart;
if (exist) {
updatedCart = cart.map((x) =>
x._id === product._id ? { ...x, qty: x.qty + 1 } : x
);
} else {
updatedCart = [...cart, { ...product, qty: 1 }];
}
setCart(updatedCart);
localStorage.setItem("cartItems", JSON.stringify(updatedCart));
};
return (

<h2>All Products</h2>

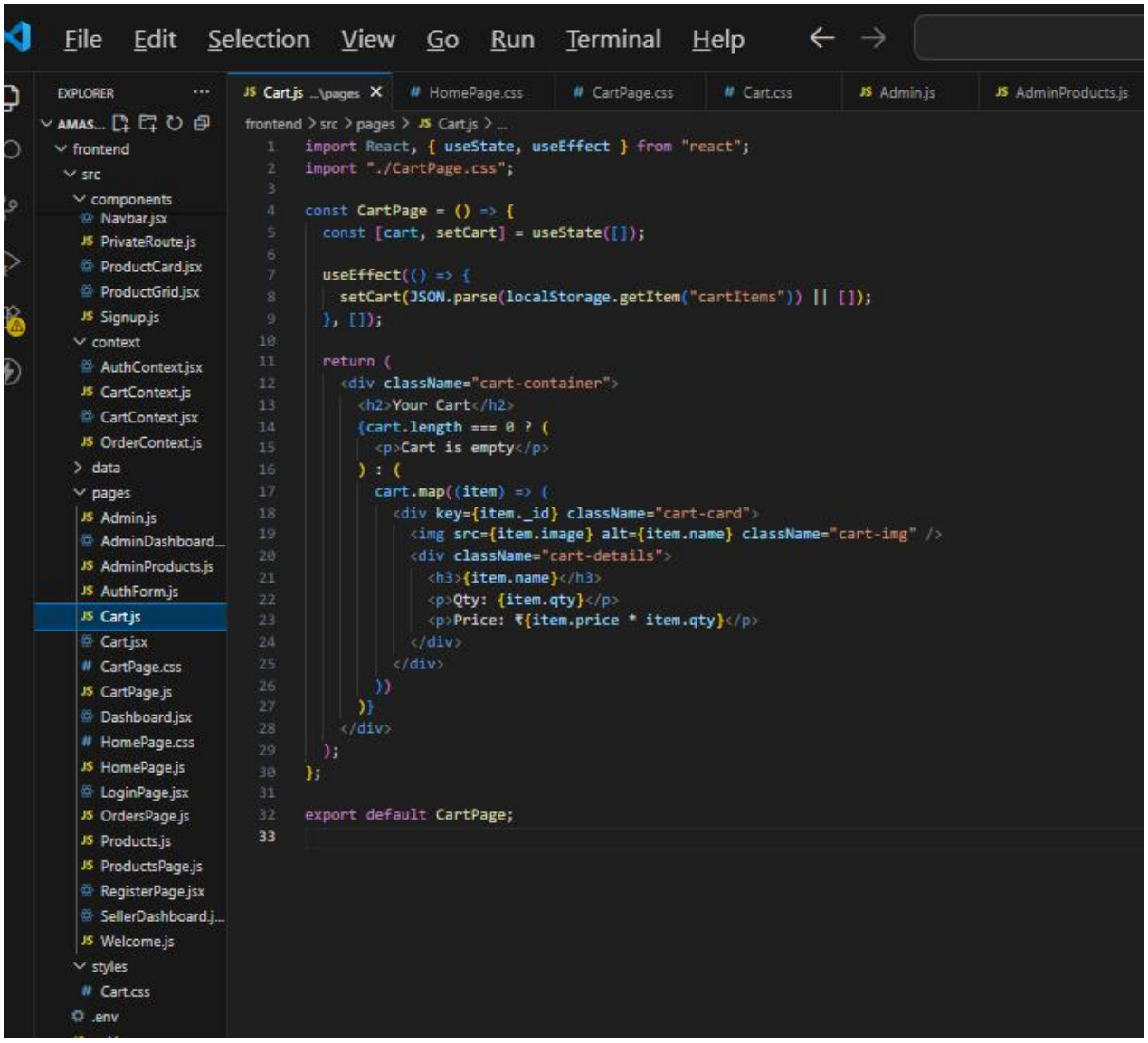
{products.length === 0 ? (
No products found
) : (
products.map((product) => (

<h3>{product.name}</h3>
{product.description}
₹{product.price}

<button onClick={() => addToCart(product)} className="add-btn">
Add to Cart
</button>

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL
Local: http://localhost:3000
On Your Network: http://10.1.1.153:3000
Note that the development build is not optimized.
To create a production build, use npm run build.
webpack compiled successfully

9.4 Cart Page


File Edit Selection View Go Run Terminal Help
EXPLORER
Cart.js ...\pages
HomePage.css
CartPage.css
Cart.css
Admin.js
AdminProducts.js
frontend > src > pages > Cart.js > ...
import React, { useState, useEffect } from "react";
import "./CartPage.css";
const CartPage = () => {
const [cart, setCart] = useState([]);
useEffect(() => {
setCart(JSON.parse(localStorage.getItem("cartItems")) || []);
}, []);
return (

<h2>Your Cart</h2>
{cart.length === 0 ? (
Cart is empty
) : (
cart.map((item) => (

<h3>{item.name}</h3>
Qty: {item.qty}
Price: ₹{item.price * item.qty}

))
)}

);
};
export default CartPage;

9.5 Seller Dashboard

frontend > src > pages > SellerDashboard.jsx > ...

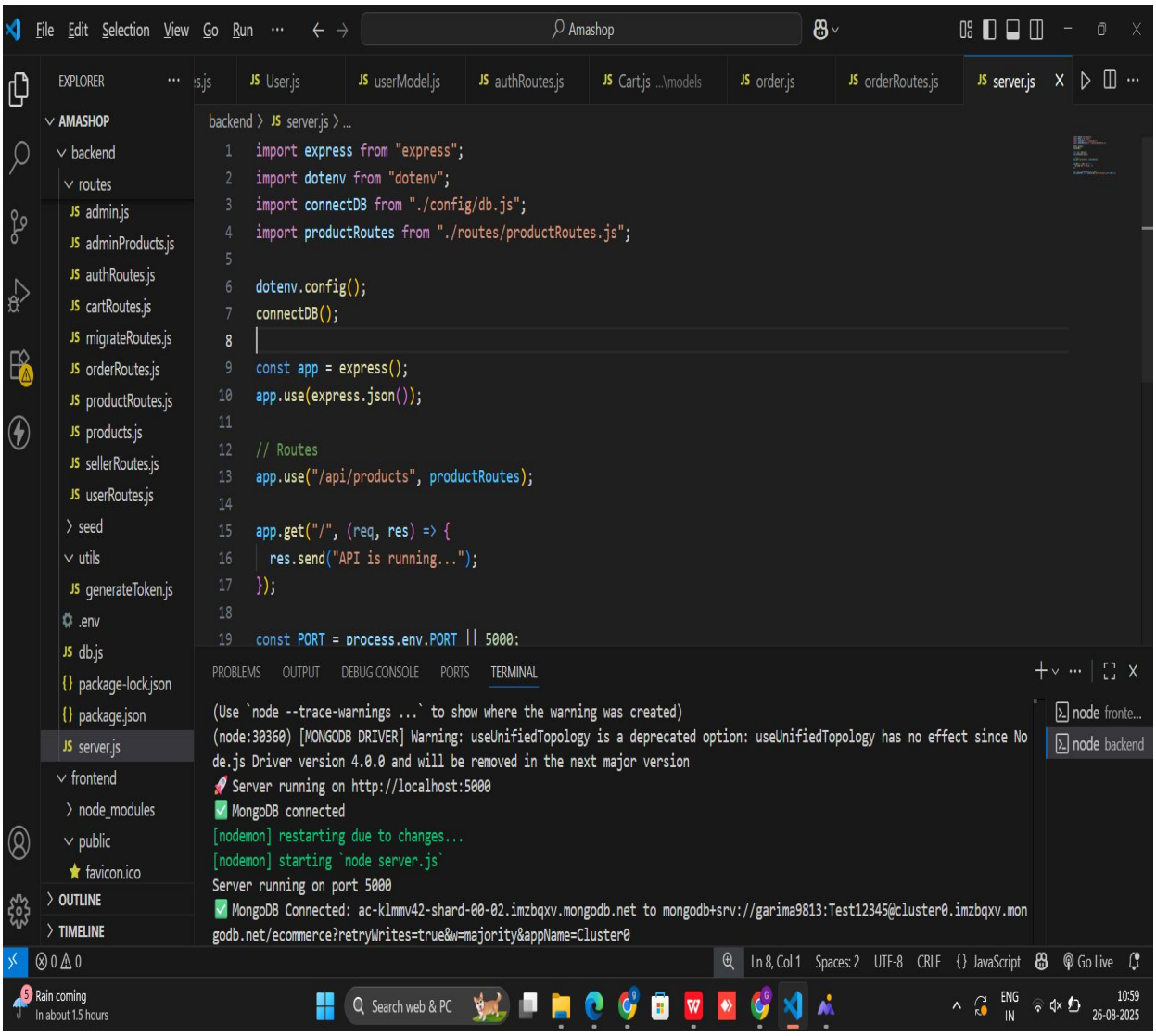
```jsx
import React, { useEffect, useState, useMemo } from "react";
import axios from "axios";

const SellerDashboard = () => {
  const [products, setProducts] = useState([]);
  const [name, setName] = useState("");
  const [description, setDescription] = useState("");
  const [price, setPrice] = useState("");
  const [image, setImage] = useState("");
  const [editingProduct, setEditingProduct] = useState(null);

  // 🟢 Token localStorage se lo
  const userInfo = JSON.parse(localStorage.getItem("userInfo"));

  // Config with token (Option 1: useMemo for clean deps)
  const config = useMemo(() => {
    return {
      headers: {
        "Content-Type": "application/json",
        Authorization: `Bearer ${userInfo?.token}`,
      },
    };
  }, [userInfo]);

  // Fetch seller's products
  useEffect(() => {
    const fetchProducts = async () => {
      try {
        const { data } = await axios.get("/api/products/mine", config);
        setProducts(data);
      } catch (error) {
        console.error("Error fetching products:", error);
      }
    };
    if (userInfo) fetchProducts();
  }, [config, userInfo]);

  // Add or Update product
  const handleAddOrUpdate = async () => {
    try {
      if (editingProduct) {
        // Update product
        const { data } = await axios.put(
          `/api/products/${editingProduct._id}`,
          { name, description, price, image },
          config
        );
        setProducts(
          products.map((p) => (p._id === data._id ? data : p))
        );
        setEditingProduct(null);
      } else {
        // Add new product
        const { data } = await axios.post(
          "/api/products",
          { name, description, price, image },
```

PROBLEMS OUTPUT DEBUG CONSOLE PORTS TERMINAL

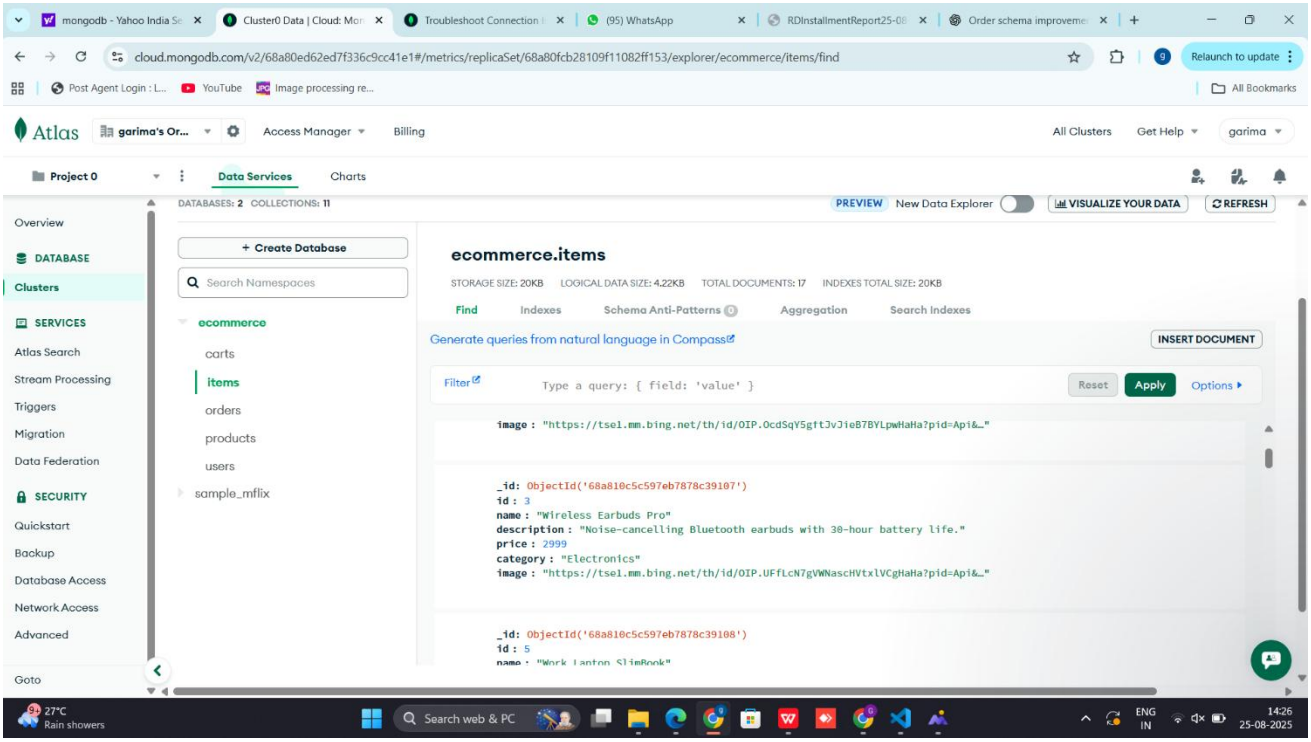
```
  Local:            http://localhost:3000
  On Your Network:  http://10.1.1.153:3000

Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
```

9.6 Admin Dashboard

9.7Backend Folder



9.8 DataBase


cloud.mongodb.com/v2/68a80ed62ed7f336c9cc41e1#/metrics/replicaSet/68a80fcb28109f11082ff153/explorer/ecommerce/items/find
Atlas
Access Manager
Billing
All Clusters
Get Help
Project 0
Data Services
Charts
Overview
DATABASE
Clusters
SERVICES
Atlas Search
Stream Processing
Triggers
Migration
Data Federation
SECURITY
Quickstart
Backup
Database Access
Network Access
Advanced
Goto
+ Create Database
Search Namespaces
ecommerce
carts
items
orders
products
users
sample_mflix
ecommerce.items
STORAGE SIZE: 20KB
LOGICAL DATA SIZE: 4.22KB
TOTAL DOCUMENTS: 17
INDEXES TOTAL SIZE: 20KB
Find
Indexes
Schema Anti-Patterns
Aggregation
Search Indexes
Generate queries from natural language in Compass
INSERT DOCUMENT
Filter
Type a query: { field: 'value' }
Reset
Apply
Options
image : "https://tse1.mm.bing.net/th/id/OIP.OcdSqY5gftJvJieB7BYLpwHaHa?pid=Api&…"
_id: ObjectId('68a810c5c597eb7878c39107')
id : 3
name : "Wireless Earbuds Pro"
description : "Noise-cancelling Bluetooth earbuds with 30-hour battery life."
price : 2999
category : "Electronics"
image : "https://tse1.mm.bing.net/th/id/OIP.UFfLcN7gVWNascHVtxlVCgHaHa?pid=Api&…"
_id: ObjectId('68a810c5c597eb7878c39108')
id : 5
27°C
Rain showers
Search web & PC
ENG
IN
14:26
25-08-2025

Chapter 10
Maintenance

10.1 Introduction

Software maintenance is one of the most crucial phases of the software development life cycle. Once a system is developed and deployed, it is not the end of the process; rather, it is the beginning of continuous improvement and adaptation. For the *Amashop* e-commerce project, maintenance ensures that the website remains functional, secure, and aligned with changing user requirements. Maintenance activities involve fixing issues, updating features, improving performance, and ensuring compatibility with evolving technologies. Since *Amashop* includes multiple panels (User, Seller, and Admin), regular maintenance is vital to keep all modules synchronized and error-free.

Key objectives of software maintenance are:

- Correct Faults: Identifying and fixing errors, bugs and defects to keep the software reliable and stable.
- Improve Performance: Optimizing speed, resource usage and system efficiency to meet evolving user expectations and system requirements.
- Adaptation: Modifying the software to work with new hardware, operating systems, or other software, ensuring compatibility with technological changes.
- Enhancement: Incorporating user feedback and adding new features or functionalities to increase the usefulness and value of the software.
- Security Updates: Addressing software vulnerabilities and implementing security patches to protect the system and its data.
- Compliance: Ensuring that the software remains in line with updated laws, regulations, and industry standards as changes occur.
- Documentation: Keeping technical documentation, manuals, and specifications updated for easier maintenance and effective user support.

10.2 Types of Maintenance

In a software lifetime, type of maintenance may vary based on its nature. It may be just a routine maintenance tasks as some bug discovered by some user or it may be a large event in itself based on maintenance size or nature. Following are some types of maintenance based on their characteristics:

- **Corrective Maintenance:** This includes modifications and updates done in order to correct or fix problems, which are either discovered by user or concluded by user error reports.
- **Adaptive Maintenance:** This includes modifications and updates applied to keep the Application product up-to date and tuned to the ever changing world of technology and business environment.
- **Perfective Maintenance -** This includes modifications and updates done in order to keep the application usable over long period of time. It includes new features, new user requirements for refining the application and improve its reliability and performance.
- **Preventive Maintenance -** This includes modifications and updations to prevent future problems of the application. It aims to attend problems, which are not significant at this moment but may cause serious issues in future.

10.3 Cost of Maintenance

The cost of maintenance depends on various factors such as system complexity, frequency of updates, and resource allocation. In the case of *Amashop*, since it is built on the MERN stack, the overall maintenance cost is relatively low compared to traditional monolithic systems. Costs may include hosting charges, domain renewal, bug fixing, security patching, and enhancements based on new requirements. Over time, maintenance may require a dedicated team if the system is scaled to handle more users, sellers, and product transactions.

10.4 Maintenance Activities

Software maintenance involves a structured set of activities to keep a software system reliable, efficient, and up to date.

The main maintenance activities are:

- **Identification & Tracing**

Detecting the need for maintenance through user reports, error logs, or automated monitoring.

Classifying the type and urgency of the required maintenance.

- **Analysis**

Evaluating the impact of the proposed modification on the system, including potential safety and security risks.

Estimating the resources, time, and cost required for the maintenance.

- **Design**

Planning new or modified components based on requirements.

Creating test cases and strategies for the proposed changes.

- **Implementation**

Actual coding and integration of the modifications.

Conducting unit testing to ensure the change is properly implemented.

- **System & Acceptance Testing**

System testing: Integrating new or modified components with the entire system and performing regressive tests to ensure system integrity.

Acceptance testing: Verifying that the maintenance meets user requirements and expectations.

- **Delivery & Deployment**

Deploying the updated system to users, either via updates or fresh installation.

Providing updated documentation, training, or user manuals as needed

10.5 Types of Software Maintenance Activities

Software maintenance activities can be broadly divided into different tasks that ensure the system's long-term sustainability:

- **Bug Fixing** – Identifying and correcting errors in functionality, UI, or database connectivity.
- **Enhancements** – Adding new features such as payment gateways, mobile app support, or product recommendations.
- **Refactoring** – Improving the code structure without changing functionality to enhance readability and maintainability.
- **Security Updates** – Regularly patching vulnerabilities and updating authentication/authorization mechanisms to prevent cyber threats
- **Database Management** – Maintaining indexes, removing redundant data, and ensuring data integrity in the MongoDB database.
- **System Upgrades** – Migrating to new versions of frameworks, tools, and operating systems for better performance and scalability.

Chapter 11
Future Scope & Conclusion

11.1 Future Scope

The *Amashop* e-commerce website has been designed with the aim of providing users with a simple and efficient online shopping experience. However, like every software system, there is always room for enhancement and growth. In the future, the system can be expanded to include a **secure online payment gateway** so that customers can directly make purchases instead of only adding products to their cart. A **review and feedback system** can also be integrated, allowing users to rate products and share their experiences, which will improve the overall trustworthiness of the platform. Furthermore, the project can be extended by developing a **mobile application version** for Android and iOS to provide better accessibility and reach a wider audience.

For the seller panel, additional features such as **sales analytics, inventory management, and automated stock alerts** can be added to assist sellers in managing their products more effectively. In the admin panel, features like **advanced user analytics, fraud detection, and AI-powered recommendations** can be introduced to improve overall business efficiency. With advancements in technologies such as Artificial Intelligence, Machine Learning, and Cloud Computing, the project can be upgraded to include **personalized product recommendations** and **scalable cloud-based hosting** to handle larger user bases.

11.2 Conclusion

In conclusion, the *Amashop* e-commerce project successfully demonstrates the development of a web-based shopping platform that provides basic yet essential functionalities such as user login/logout, product browsing, adding items to the cart, and managing them efficiently. The project also includes a seller panel and an admin panel, making it a complete three-tier system that addresses the needs of users, sellers, and administrators. Through this project, the use of **MERN stack technologies** (MongoDB, Express.js, React.js, Node.js) has been effectively implemented to build a secure, interactive, and scalable application.

References

- **Visual Studio Code (VS Code):** A free and lightweight code editor used for development. https://code.visualstudio.com/
- **HTML, CSS, JavaScript:** The core technologies for building the frontend user interface. https://www.w3schools.com/howto/default_page5.asp
- **Node.js:** A JavaScript runtime environment used for building the backend. https://nodejs.org/en
- **Express.js:** A minimal and flexible Node.js web application framework. https://expressjs.com/
- **Mongo DB:** MongoDB is a NoSQL, document-oriented database that stores data in flexible JSON-like documents—see the official docs: https://www.mongodb.com/docs/